2015 IEEE Symposium on Service-Oriented System Engineering

A Reusable Automated Acceptance Testing Architecture for Microservices in Behavior-Driven Development

Mazedur Rahman
Member IEEE, San Jose State University, USA
mazedur.rahman.liton@gmail.com

Jerry Gao
San Jose State University, USA
jerry.gao@sjsu.edu

***Abstract* – Cloud Computing and Mobile Cloud Computing are reshaping the way applications are being developed and deployed due to their unique needs such as massive scalability, guaranteed fault tolerance, near zero downtime, etc. and also daunting challenges such as security, reliability, continuous deployment and update capability. Microservices architecture, where application is composed of a set of independently deployable services, is increasingly becoming popular due to its capability to address most of these needs and challenges. In recent years, the Behavior-Driven Development (BDD) has become one of the most popular agile software development processes, and frequently used in microservices development. The key to success of BDD is the executable acceptance tests that describe the expected behavior of a feature and its acceptance criteria in the form of scenarios using simple and business people readable syntax. The reusability, auditability, and maintainability become some of the major concerns when BDD test framework is applied for each microservice repository and no previous research addresses these concerns. In this paper, we present a reusable automated acceptance testing architecture to address all these concerns.**

Keywords – executable automated acceptance testing; Gherkin; functional testing; behavior-driven development; microservice

I. INTRODUCTION

In today's software world, driven by agile development processes, the applications also need to be very nimble. These applications need to be fast developed to address changing business need, instantly deployed to allow continuous deployment, and scaled entire or part of the applications enormously. They may need to cater support for diverse clients such as desktop browsers, mobile browsers, native mobile applications as well as third parties through APIs. Especially the server-side of the application needs to be very agile to address these varied needs. Meeting all these simultaneously by a monolithic application has become incredibly difficult and in some cases impossible. This has led to an architectural shift from monolithic to microservices style where an application is composed of a group of services that can be deployed independently.

The Behavior-Driven Development (BDD) [1], introduced by Dan North and inspired by Kent Beck's Test-Driven Development (TDD) [2], is an outside-in agile software development process frequently used in Extreme Programming [3] and microservices development. BDD uses a business readable and Domain Specific Language (DSL) [4] called Gherkin [5] to describe software's behaviors while hiding its implementation details. In BDD, acceptance test scenarios are explicitly written down with the following simple syntax [1, 6, 7]:

Given some initial context,
When an event occurs,
Then ensure some outcomes.

In BDD, software requirements or behaviors are initially gathered using scenarios with the above format where the lines with "*Given*", "*When*" and "*Then*" keywords are known as "*Step*". Each of these steps are parsed and executed by a BDD test framework to verify the software's expectation, also known as Automated Acceptance Tests (AAT). These scenarios act as a powerful means of communication among software developers, testers, and analysts to improve the quality of the software. They also work as the living documentation of the expected behaviors of the software.

Agile software development methodologies are growing in popularity in the software industry due to their higher chances to succeed [8, 9, 10]. TDD, BDD, Acceptance Test-Driven Development (ATDD) are examples of widely used agile development processes where acceptance testing plays a massive role in their successes. Among these, there has been a growing interest for BDD due to its capability to engage business analyst to gather requirements and easily converting those requirements into business people readable but executable scenarios that work as automated acceptance tests [1]. BDD is being embraced by more and more software development teams recently.

While automated acceptance testing is the precursor to BDD's success, it has its own challenges. The automated acceptance tests are usually black-box system tests that may heavily rely on expensive fixtures in databases, file systems and network communications. These tests are generally very slow compared to unit tests [11, 12] and often suffer from non-determinism [13] due to their extensive fixture dependency. While unit test targets the smallest program logic and hence remains stable, acceptance test targets capturing software's business expectation that changes frequently and hence needs ongoing update, refactor and maintenance. Though some IDE based refactoring tools exist for unit tests, and Fit/FitNesse based acceptance tests such as ReFit [14], no such tools yet to exist for acceptance tests in BDD. This makes maintenance of these acceptance tests even harder. Thus, a careful design of acceptance tests keeping refactoring and reusability in mind becomes very crucial for BDD acceptance tests. This becomes even more critical for an application built based on microservices

978-1-4799-8356-8/15 $31.00 © 2015 IEEE
DOI 10.1109/SOSE.2015.55

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:34:44 UTC from IEEE Xplore. Restrictions apply.

architecture [15, 16] where often same BDD test framework along with common BDD "steps" for scenarios used in multiple repositories for microservices as shown in Figure 1.

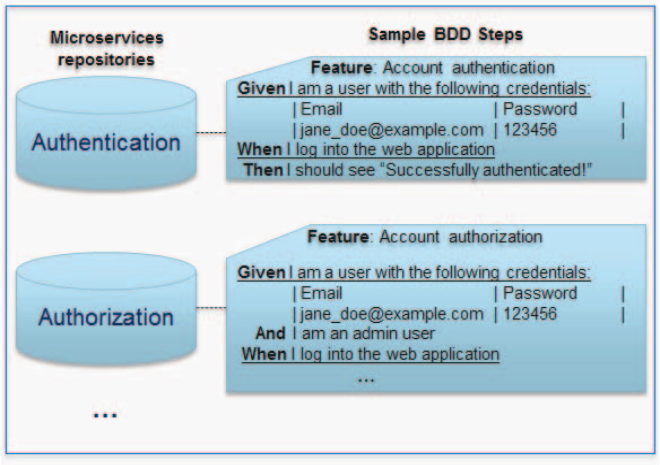

Figure 1. Sample BDD steps from a web application composed of Microservices Architecture

The underlined "*Given*" and "*When*" steps in Figure 1 are identical in both "Authentication service" and "Authorization service" repositories. Since these tests along with the BDD test framework are part of separate source code repositories, there is no way to reuse the implementation of same "steps" in one repository by the features from the other repositories when BDD test framework executes them. Thus, it becomes even harder to refactor common "steps" across multiple repositories. To address these concerns, we present a reusable architecture for the acceptance tests in Section III.

Rest of the paper is organized as follows. Section II identifies the major needs and challenges for reusable acceptance tests in BDD, Section III introduces the reusable automated acceptance architecture, Section IV outlines future work, and finally conclusive remarks are given in Section V.

II. Needs and Challenges

A. Needs

The major needs for microservices development using BDD process are listed below:

- **Reusability of step implementations for BDD scenarios**: As a software developer/tester, I want to reuse the implementation of common steps for BDD scenarios used by multiple microservices, so that I can avoid duplicate code for each implementation in multiple codebases for those services. For example if a BDD step "*I am a user with the following credentials*" is used by two microservices (see Figure 1) – "*Account Authentication Service*" and "*Account Authorization Service*", then the implementation for this step should be reusable for both services.
- **Separation of concern among developers, testers and business analysts**: As a developer/tester/business analyst, I want to keep my BDD scenarios (that work as acceptance tests) decoupled from application source code to allow development and testing teams to iterate independently on them while working with the business analysts when appropriate. Often for big enterprise application, there are separate development and testing teams. They collaborate from the beginning of feature and story requirement analysis, refinement and identifying acceptance criteria. In some cases the testing team focuses on making acceptance tests scenarios executable as black-box tests, and may iterate on them independently. Initially developers and/or testers work with business analysts to come up with BDD scenarios while gathering the requirements. Usually these scenarios are refined iteratively multiple times even before the development starts. Thus each refinement activity needs to be tracked using a version control system accessible to developers, testers and business analysts. But source control repository is often owned by developers, and testers/analysts may not have same access privilege that makes tracking these activities difficult.
- **Ease of auditability**: As a product owner/stakeholder/business analyst, I want to keep acceptance tests in a common repository, so that I can easily be able to audit the capability of my application built on microservices architecture. Often a microservices architecture based application is composed of many microservices. If a lot of BDD features/scenarios are spread across large number of microservices repositories, it becomes difficult for the product owners/stakeholders/business analysts to know the capability of the application as a whole.

B. Challenges

Here are the major challenges of applications developed in BDD following microservices architecture:

- **Maintainability**: The major maintenance overhead for BDD acceptance tests for microservices involves the following area:
  - **BDD framework version upgrade**: Usually BDD acceptance tests are implemented using a standard BDD acceptance test framework. Cucumber [19], Behat [20], and Behave [21] are such BDD frameworks for Ruby, PHP and Python programming languages respectively. If the same BDD framework is used for multiple microservices to implement BDD scenarios, then upgrading the version of the framework may be increased dramatically if it involves a larger number of microservice repositories. This can be very time consuming, tedious and counterproductive.
  - **Refactoring**: Agile development processes embrace changing business need which is inevitable during the life cycle of the most software products. Thus if the BDD step definition changes to reflect new business policies, then refactoring those steps for many microservices across multiple repositories will result in duplicate effort and impede the application development speed significantly.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:34:44 UTC from IEEE Xplore. Restrictions apply.

- **Systems integration complexity during runtime**: Unlike monolithic architecture based application (where systems integration generally happens during build time), systems or components integration for microservices architecture based application happens during runtime. This integration complexity shift from build to runtime phase increases the risk of failures in production environment. To complete a business workflow, microservices based application may require remote calls to multiple services increasing the point of failures. This risk can be reduced by having higher number of acceptance tests that exercise systems cohesiveness. For example if service A calls service B, and service B calls C to fulfill a business workflow, the acceptance tests should subsume behaviors from each of these services as much as possible.
- **Black-box testing using production-like execution environment**: While TDD encourages writing unit tests in isolation with the help of mocks, stubs, fakes, etc., BDD encourages writing end-to-end acceptance tests reflecting business behavior with real life execution environment during development phase. Emulating production-like execution environment during development and testing can be daunting considering complex need such as scalability, fault tolerance, logging, monitoring, and so on.

III. The Reusable Automated Acceptance Testing Architecture

A. Terminology and Assumption

Here are the major terminologies and their meaning used in this paper:

- **Automated Acceptance Tests (AAT)**: Automated acceptance tests, also referred as "customer tests" or "story tests", assert the expectations or requirements of software product based on usage scenarios and acceptance criteria [7]. They help to validate the software requirement specified by contract [17].
- **Monolithic Architecture**: In a monolithic architecture, an application is built as a single unit, and logic to handle a request runs in a single process [15]. If part of application is changed, then entire application needs to be rebuilt and redeployed. Usually systems integration for a monolithic application happens during build time.

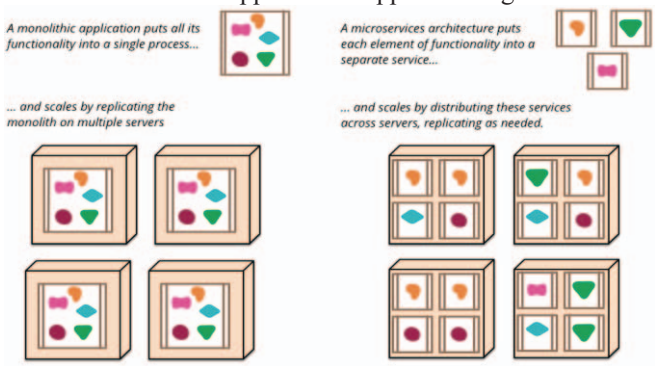

Figure 2. Monolithic and Microservices based architecture [15]

- **Microservices Architecture**: Generally microservices architecture based application is composed of a set of independently deployable services [15]. These services are tracked by separate repositories using a version control system such as git, mercurial, subversion, etc. Each service may be developed and deployed independently by different teams as shown in Figure 2. Often they tend to follow single responsibility principle (SRP), i.e. to solve one business need nicely. These services can be run in different production server nodes and they usually communicate with each other through remote procedure calls or as a web service request. Usually systems integration for the composed application happens during runtime.

B. Problem Statement

Let's consider two sample microservices of a web application named "*example.com*" – "Account Authentication Service", "Account Authorization Service", and they are tracked by two repositories using a distributed version control system such as git. Now let's take two example BDD scenarios, one from each service as shown in Figure 3 and 4.

```
Feature: Account authentication
  Scenario: User can be authenticated with valid credentials
    Given I am a user with the following credentials:
      | Email                | Password  |
      | jane_doe@example.com | 123456    |
    When I log into the web application
    Then I should see "Successfully authenticated!"
```

Figure 3. An example BDD scenario from Account Authentication Service

```
Feature: Account authorization
  Scenario: Admin can see account management controls
    Given I am a user with the following credentials:
      | Email                | Password  |
      | jane_doe@example.com | 123456    |
      And I am an admin user
    When I log into the web application
      And I click "example.com/accounts"
    Then I should see "Some account management controls"
```

Figure 4. An example BDD scenario from Account Authorization Service

The BDD steps with "*Given*" and "*When*" keywords are identical for both services and hence the implementations for these steps will also be identical. The step with "*Then*" keyword is also identical with the exception of the passed content of the parameter inside the quotations. Thus implementation for this step will also be identical. Each of these steps should have only one implementation for both repositories to avoid duplicate code, to allow easier refactoring, and to reflect same business policy for both services.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:34:44 UTC from IEEE Xplore. Restrictions apply.

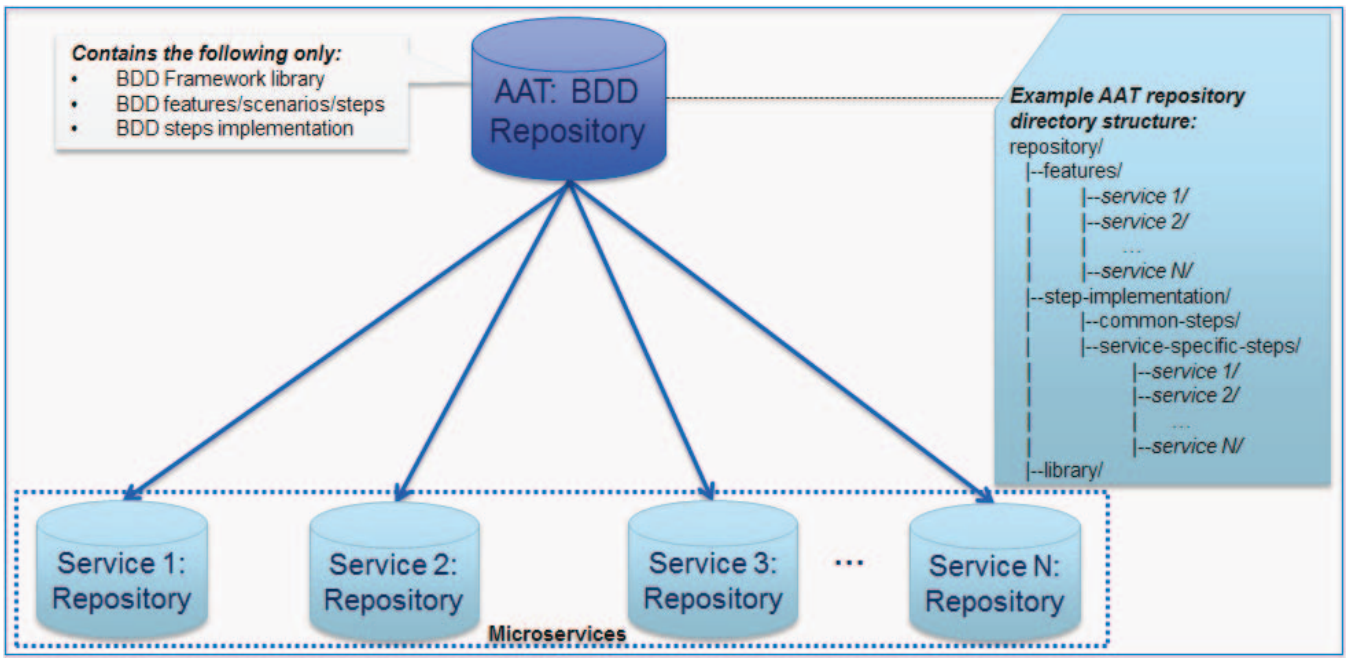

Figure 5. Reusable Automated Acceptance Testing Architecture (RAATA)

C. *System Architecture*

We introduce Reusable Automated Acceptance Testing Architecture (RAATA) to address the needs, challenges and problems presented in section II-A, II-B and III-B respectively. Instead of keeping BDD features/scenarios based automated acceptance tests spread across multiple microservices repositories of an application, we introduce a new repository called Automated Acceptance Tests (AAT) repository as shown in Figure 5. Typically an AAT repository has the following top level directories:

- **features**: This directory contains one subdirectory for each microservice. If an application consists of *N* microservices, then there will be *N* subdirectories starting from *service 1* through *service N*. Each microservice subdirectory contains all the feature files with BDD scenarios necessary for that service. Organizing all the features of an application this way allows easy auditing of the capabilities of the application.
- **step-implementation**: This directory contains implementations for the BDD steps recorded in the subdirectory under features directory. The implementations for the common steps across two or more microservices for the application are placed under a subdirectory called "common-steps" and service specific steps are placed under subdirectories named after each microservice. Implementing BDD steps this way with the AAT repository helps to avoid duplicate code for the same step. This also allows easy refactoring of the step implementation as each step has only one implementation, and hence only one place to refactor the implementation.
- **library**: This directory contains the source code for the selected BDD framework. Since the same framework is applied to all microservices, upgrading the version of the framework becomes a very easy and trivial task.

Since all the behaviors of an application composed of microservices are managed using a single repository, collaboration among analysts, testers, developers, product owners and stakeholders becomes very convenient and simple. This helps to increase cohesiveness of the microservices based application as well as improves the quality of the product due to better collaboration. AAT repository also decouples the end-to-end acceptance tests cleanly from the source code of the microservices. This way the acceptance tests become completely black-box and hence they represent production-like behavior better unlike TDD where mock objects are used to isolate dependent modules [18]. Also the separate AAT repository allows the testers to iterate on building end-to-end BDD scenario based system tests, while developers can focus on implementation of the feature using the source control repositories dedicated for microservices. This approach also helps to avoid the conflict between developers and testers.

D. *Challenges*

While our proposed automated acceptance testing architecture improves auditability of BDD steps, reusability of their implementations, and provides separation of concerns among developers, testers and business analysts, keeping the acceptance tests synchronized with service repositories can be challenging. In BDD process, the business behaviors or expectations of

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:34:44 UTC from IEEE Xplore. Restrictions apply.

a feature are captured in scenarios and steps. Similar to tests first approach of TDD, BDD encourages capturing software behaviors first. BDD follows the same red-green-refactor cycle of TDD while emphasizing on the 'behavior' instead of 'test', as shown in Figure 6. The automated acceptance test written in BDD fails initially, but passes once the implementation fulfills the expected behaviors. Thus, enforcing the BDD principle properly, we can reduce the chances of our proposed acceptance test repository being out of synchronization with the service repositories.

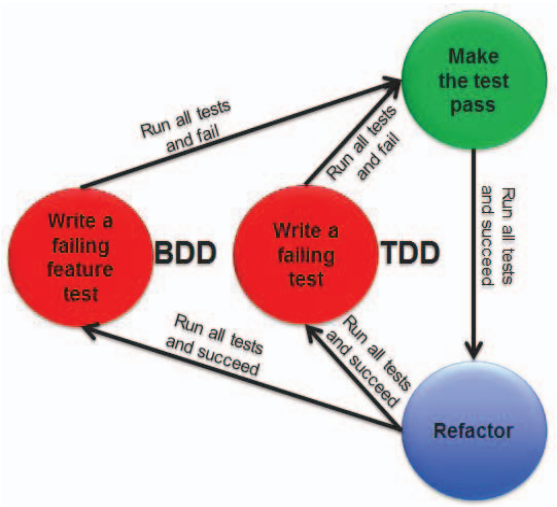

Figure 6. Red-Green-Refactor cycle during TDD and BDD

IV. Future Work

In the future, we intend to evaluate our proposed reusable automated acceptance testing architecture using applications composed of complicated microservices and more complex test cases. We would like to conduct more quantitative studies on how this framework helps to improve collaboration among business analysts, testers, developers and product owners by reducing the maintenance burden of the acceptance tests. We will invite software development teams to apply our framework and measure the impact of this framework on quality of software. In addition, we would like to experiment the ease of refactoring of the BDD steps based on some existing refactoring tools.

V. Conclusions

The automated acceptance testing is regarded as one of the most effective means of building robust and bug-free software satisfying the customer expectations. But software development teams often refrain from applying it to microservices due to its high maintenance cost and fail to take advantage of this powerful technique. Our proposed architecture to reuse BDD acceptance tests across multiple repositories will take a lot of the maintenance burden away from the developers/testers and will encourage them to apply BDD acceptance tests more frequently. In addition, this architecture will also minimize issues with auditability, and systems integration maintenance. This will also reduce the conflict between developers and testers by allowing them to independently iterate on separate repositories while achieving the same application goals.

References

[1] D. North "Introducing behaviour driven development", (2006), Available online at: http://dannorth.net/introducing-bdd/. Accessed on Sep 20, 2014.

[2] K. Beck, Test-Driven Development: by Example. Addison Wesley, 2003.

[3] K. Beck, C. Andres, Extreme Programming Explained, 2nd Edition, Addison-Wesley, 2005.

[4] M. Fowler, "BusinessReadableDSL", Available online at: http://martinfowler.com/bliki/BusinessReadableDSL.html, Dec 15, 2008. Retrieved on Nov 1, 2014.

[5] Gherkin. Available online at: http://cukes.info/gherkin.html, Retrieved on Nov 1, 2014.

[6] Gherkin. Available online at: https://github.com/cucumber/cucumber/wiki/Gherkin, Retrieved on Nov 1, 2014.

[7] Acceptance Testing. Available online at: http://guide.agilealliance.org/guide/acceptance.html, Retrieved on Nov 1, 2014.

[8] M. Fowler, "Agile Software Development", Available online at: http://martinfowler.com/agile.html. Retrieved on Sep 20, 2014.

[9] S. Ambler, "2013 IT Project Success Rates Survey Results", Available online at: http://www.ambysoft.com/surveys/success2013.html. Retrieved on Sep 20, 2014.

[10] Murphy, B., et al.. Have Agile Techniques been the Silver Bullet for Software Development at Microsoft?, Empirical Software Engineering and Measurement, ACM / IEEE International Symposium,(2013), 75—84. 2013.

[11] R. Osherove. The art of unit testing: with examples in.NET. Greenwich (Connecticut): Manning. 2009.

[12] Unit Testing, Avaialable online at: http://guide.agilealliance.org/guide/unittest.html, Retrieved on Nov 1, 2014.

[13] M. Fowler, "Eradicating Non-Determinism in Tests", Available online at: http://martinfowler.com/articles/nonDeterminism.html, Retrieved on Nov 1, 2014.

[14] R. Borg and M. Kropp. Automated acceptance test refactoring. In Proceedings of the 4th Workshop on Refactoring Tools, WRT '11, pages 15–21, New York, NY, USA, 2011.

[15] M. Fowler, "Microservices", Available online at: http://martinfowler.com/articles/microservices.html, Retrieved on Nov 2, 2014.

[16] Pattern: Microservices Architecture. Available online at: http://microservices.io/patterns/microservices.html, Retrieved on Nov 2, 2014.

[17] J. Marciniak. Encyclopedia of software engineering. New York: Wiley, 1994

[18] Mock Objects. Available online at: http://guide.agilealliance.org/guide/mocks.html, Retrieved Nov 2, 2014.

[19] Cucumber. Available online at: http://cukes.info/

[20] Behat Documentation. Available online at: http://docs.behat.org/en/v2.5/

[21] Behave. Available online at: http://pythonhosted.org/behave/

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:34:44 UTC from IEEE Xplore. Restrictions apply.